AI LAB FAT

1) BFS:

```
from collections import deque
graph = {}
# Number of nodes
n = int(input("Enter number of nodes: "))

# Taking adjacency list input
for i in range(n):
    node = input("Enter node: ")
    neighbors = input("Enter neighbors (space-separated): ").split()
    graph[node] = neighbors

# Start node
start = input("Enter starting node: ")
visited = set()
queue = deque()

# BFS
queue.append(start)
visited.add(start)

print("BFS Traversal:")

while queue:
```

```
    node = queue.popleft()
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            queue.append(neighbor)
            visited.add(neighbor)
```

```
Bash

Enter number of nodes: 4
Enter node: A
Enter neighbors (space-separated): B C
Enter node: B
Enter neighbors (space-separated): D
Enter node: C
Enter neighbors (space-separated):
Enter node: D
Enter neighbors (space-separated):
Enter starting node: A
BFS Traversal:
A B C D
```

2) DFS:

```
graph = {}

# Number of nodes
n = int(input("Enter number of nodes: "))

# Taking adjacency list input
for i in range(n):
    node = input("Enter node: ")
    neighbors = input("Enter neighbors
(space-separated): ").split()
    graph[node] = neighbors
```

```python
# Start node
start = input("Enter starting node: ")

visited = set()

def dfs(node):

    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)

print("DFS Traversal:")
dfs(start)
```

Bash

```
Enter number of nodes: 4
Enter node: A
Enter neighbors (space-separated): B C
Enter node: B
Enter neighbors (space-separated): D
Enter node: C
Enter neighbors (space-separated):
Enter node: D
Enter neighbors (space-separated):
Enter starting node: A
DFS Traversal:
A B D C
```

3) IDS:

```
graph = {}

# Number of nodes
n = int(input("Enter number of nodes: "))

# Taking adjacency list input
for i in range(n):
    node = input("Enter node: ")
    neighbors = input("Enter neighbors
(space-separated): ").split()
    graph[node] = neighbors

start = input("Enter start node: ")
goal = input("Enter goal node: ")
max_depth = int(input("Enter maximum depth: "))

# Depth Limited Search
def dls(node, goal, depth):
```

```
    if node == goal:
        return True
    if depth <= 0:
        return False

    for neighbor in graph[node]:
        if dls(neighbor, goal, depth - 1):
            return True
    return False

# Iterative Deepening Search
def ids(start, goal, max_depth):
    for depth in range(max_depth + 1):
        print("Checking at depth:", depth)
        if dls(start, goal, depth):
            print("Goal found at depth:", depth)
            return
    print("Goal not found")

ids(start, goal, max_depth)
```

```
Bash

Enter number of nodes: 4
Enter node: A
Enter neighbors (space-separated): B C
Enter node: B
Enter neighbors (space-separated): D
Enter node: C
Enter neighbors (space-separated):
Enter node: D
Enter neighbors (space-separated):

Enter start node: A
Enter goal node: D
Enter maximum depth: 3
Checking at depth: 0
Checking at depth: 1
Checking at depth: 2
Goal found at depth: 2
```

4) UCS:

```
import heapq

graph = {}

# Number of nodes
```

```
n = int(input("Enter number of nodes: "))

# Input graph with costs
for i in range(n):
    node = input("Enter node: ")
    edges = input("Enter neighbors with cost
(format: B-2 C-3): ").split()

    temp = []
    for e in edges:
        neighbor, cost = e.split('-')
        temp.append((neighbor, int(cost)))

    graph[node] = temp

start = input("Enter start node: ")
goal = input("Enter goal node: ")

# UCS
def ucs(start, goal):
    pq = []
    heapq.heappush(pq, (0, start))  # (cost, node)

    visited = set()

    while pq:
        cost, node = heapq.heappop(pq)
```

```
        if node in visited:
            continue

        print(node, "Cost:", cost)
        visited.add(node)

        if node == goal:
            print("Goal reached with cost:", cost)
            return

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                heapq.heappush(pq, (cost + weight,
neighbor))

    print("Goal not found")

ucs(start, goal)
```

Bash

```
Enter number of nodes: 4
Enter node: A
Enter neighbors with cost (format: B-2 C-3): B-2 C-3
Enter node: B
Enter neighbors with cost (format: B-2 C-3): D-3
Enter node: C
Enter neighbors with cost (format: B-2 C-3):
Enter node: D
Enter neighbors with cost (format: B-2 C-3):

Enter start node: A
Enter goal node: D
A Cost: 0
B Cost: 2
C Cost: 3
D Cost: 5
Goal reached with cost: 5
```

5) GBFS:

```
import heapq

graph = {}
heuristic = {}

# Number of nodes
n = int(input("Enter number of nodes: "))
```

```
# Input graph
for i in range(n):
    node = input("Enter node: ")
    neighbors = input("Enter neighbors
(space-separated): ").split()
    graph[node] = neighbors

# Input heuristic values
print("Enter heuristic values:")
for i in range(n):
    node = input("Node: ")
    h = int(input("Heuristic value: "))
    heuristic[node] = h

start = input("Enter start node: ")
goal = input("Enter goal node: ")

# Greedy Best First Search
def gbfs(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], start))

    visited = set()

    while pq:
        h, node = heapq.heappop(pq)

        if node in visited:
```

```
            continue

        print(node, "h(n):", h)
        visited.add(node)

        if node == goal:
            print("Goal reached!")
            return

        for neighbor in graph[node]:
            if neighbor not in visited:
                heapq.heappush(pq,
(heuristic[neighbor], neighbor))

    print("Goal not found")

gbfs(start, goal)
```

‹/› Bash

```
Enter number of nodes: 4
Enter node: A
Enter neighbors (space-separated): B C
Enter node: B
Enter neighbors (space-separated): D
Enter node: C
Enter neighbors (space-separated):
Enter node: D
Enter neighbors (space-separated):

Enter heuristic values:
Node: A
Heuristic value: 5
Node: B
Heuristic value: 3
Node: C
Heuristic value: 4
Node: D
Heuristic value: 0

Enter start node: A
Enter goal node: D
A h(n): 5
B h(n): 3
D h(n): 0
Goal reached!
```

6) A* :

```
import heapq

graph = {}
heuristic = {}

# Number of nodes
n = int(input("Enter number of nodes: "))

# Input graph with costs
```

```
for i in range(n):
    node = input("Enter node: ")
    edges = input("Enter neighbors with cost (B-2
C-3): ").split()

    temp = []
    for e in edges:
        neighbor, cost = e.split('-')
        temp.append((neighbor, int(cost)))

    graph[node] = temp

# Input heuristic
print("Enter heuristic values:")
for i in range(n):
    node = input("Node: ")
    h = int(input("Heuristic: "))
    heuristic[node] = h

start = input("Enter start node: ")
goal = input("Enter goal node: ")

def astar(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], 0,
start))  # (f, g, node)

    visited = set()
```

```
    while pq:
        f, g, node = heapq.heappop(pq)

        if node in visited:
            continue

        print(node, "f(n):", f)
        visited.add(node)

        if node == goal:
            print("Goal reached with cost:", g)
            return

        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]
                heapq.heappush(pq, (new_f, new_g,
neighbor))

    print("Goal not found")

astar(start, goal)
```

‹/› Bash

```
Enter number of nodes: 4
Enter node: A
Enter neighbors with cost (B-2 C-3): B-1 C-3
Enter node: B
Enter neighbors with cost (B-2 C-3): D-3
Enter node: C
Enter neighbors with cost (B-2 C-3):
Enter node: D
Enter neighbors with cost (B-2 C-3):

Enter heuristic values:
Node: A
Heuristic: 4
Node: B
Heuristic: 2
Node: C
Heuristic: 4
Node: D
Heuristic: 0

Enter start node: A
Enter goal node: D
A f(n): 4
B f(n): 3
D f(n): 4
Goal reached with cost: 4
```

7) minmax:

```
def minimax(depth, isMax):

    # Leaf nodes (example values)
    scores = [3, 5, 2, 9]

    # If leaf node reached
    if depth == 2:
        return scores.pop(0)

    if isMax:
        return max(minimax(depth + 1, False),
                   minimax(depth + 1, False))
    else:
        return min(minimax(depth + 1, True),
                   minimax(depth + 1, True))

result = minimax(0, True)
print("Optimal value:", result)
```

```
</> Bash

Optimal value: 5
```

8) alphabeta:

```
def alphabeta(depth, nodeIndex, isMax, values,
alpha, beta):

    # Leaf node
    if depth == 2:
        return values[nodeIndex]
```

```
    if isMax:
        best = -1000

        for i in range(2):
            val = alphabeta(depth + 1, nodeIndex *
2 + i, False, values, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)

            if beta <= alpha:
                break   # PRUNE

        return best

    else:
        best = 1000

        for i in range(2):
            val = alphabeta(depth + 1, nodeIndex *
2 + i, True, values, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)

            if beta <= alpha:
                break   # PRUNE

        return best
```

```
values = [3, 5, 2, 9]
result = alphabeta(0, 0, True, values, -1000, 1000)
print("Optimal value:", result)
```

```
Bash

Optimal value: 5
```

9) Hill Climbing:

```
def hill_climbing():
    current = int(input("Enter initial value: "))

    while True:
        neighbors = list(map(int, input("Enter
neighbors: ").split()))

        best = max(neighbors)

        if best <= current:
            print("Reached peak:", current)
            break

        current = best
        print("Move to:", current)

hill_climbing()
```

```bash
Enter initial value: 5
Enter neighbors: 6 8 7
Move to: 8
Enter neighbors: 9 10 8
Move to: 10
Enter neighbors: 9 7 6
Reached peak: 10
```

10) Missionary cannibal :

```
from collections import deque

def is_valid(m, c):
    return (m == 0 or m >= c) and (3-m == 0 or 3-m
>= 3-c) and 0 <= m <= 3 and 0 <= c <= 3

def bfs():
    start = (3, 3, 1)
    goal = (0, 0, 0)

    queue = deque([(start, [])])
    visited = set()

    while queue:
        (m, c, boat), path = queue.popleft()

        if (m, c, boat) in visited:
            continue

        visited.add((m, c, boat))
        path = path + [(m, c, boat)]

        if (m, c, boat) == goal:
            return path

        moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
```

```
            for dm, dc in moves:
                if boat == 1:
                    new_state = (m-dm, c-dc, 0)
                else:
                    new_state = (m+dm, c+dc, 1)

                if is_valid(new_state[0],
new_state[1]):
                    queue.append((new_state, path))

    return None

solution = bfs()

for step in solution:
    print(step)
```

```
</> Bash

(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
```

11) water jug:

```
from collections import deque
def water_jug(x, y, target):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        a, b = queue.popleft()

        if (a, b) in visited:
            continue
```

```
            print((a, b))
            visited.add((a, b))

            if a == target or b == target:
                print("Goal reached!")
                return

            # All possible moves
            queue.append((x, b))   # fill jug1
            queue.append((a, y))   # fill jug2
            queue.append((0, b))   # empty jug1
            queue.append((a, 0))   # empty jug2

            # pour jug1 -> jug2
            pour = min(a, y - b)
            queue.append((a - pour, b + pour))

            # pour jug2 -> jug1
            pour = min(b, x - a)
            queue.append((a + pour, b - pour))

# User input
x = int(input("Enter capacity of Jug1: "))
y = int(input("Enter capacity of Jug2: "))
target = int(input("Enter target: "))

water_jug(x, y, target)
```

```
</> Bash

Enter capacity of Jug1: 4
Enter capacity of Jug2: 3
Enter target: 2
(0, 0)
(4, 0)
(4, 3)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
Goal reached!
```

12) N Queens:

```
def backtrack(row):
    global n,col,pos,neg,result,board
    if row == n:
        copy = ["".join(r) for r in board]
        result.append(copy)
        return
    for c in range(n):
        if c in col or (row+c) in pos or (row-c) in
neg:
            continue
        col.add(c)
        pos.add(row+c)
        neg.add(row-c)
        board[row][c] = "Q"
        backtrack(row+1)
        col.remove(c)
        pos.remove(row+c)
        neg.remove(row-c)
        board[row][c]="[]"
n=int(input("Enter the value of n: "))
col = set()
pos = set()
neg = set()
result = []
board =[["[]"]* n for _ in range(n)]
backtrack(0)
count=1
```

```
for i in result:
    print(f"Solution {count}:")
    for j in i:
        print(j)
    count+=1
```

```
</> Bash

Enter the value of n: 4
Solution 1:
.Q..
...Q
Q...
..Q.

Solution 2:
..Q.
Q...
...Q
.Q..
```

13) Tic Tac Toe:

```
row = 3
col = 3

tic_tack = []
for i in range(row):
    r = []
    for j in range(col):
        r.append(" ")
    tic_tack.append(r)

exit_game = True
player1_flag = True

p1 = input("Player 1, choose X or O: ").upper()
if p1 == "X":
    p2 = "O"
```

```
elif p1 == "O":
    p2 = "X"
else:
    print("Invalid choice")
    exit_game = False
```

```
def print_board():
    for i in range(row):
        print(tic_tack[i])

def check_win(player):
    # rows
    for i in range(3):
        if tic_tack[i][0] == tic_tack[i][1] ==
tic_tack[i][2] == player:
            return True

    # columns
    for j in range(3):
        if tic_tack[0][j] == tic_tack[1][j] ==
tic_tack[2][j] == player:
            return True

    # diagonals
    if tic_tack[0][0] == tic_tack[1][1] ==
tic_tack[2][2] == player:
        return True
    if tic_tack[0][2] == tic_tack[1][1] ==
tic_tack[2][0] == player:
        return True

    return False
```

```
while exit_game:
    print_board()

    if player1_flag:
        print("Player 1 Turn")
        player = p1
    else:
        print("Player 2 Turn")
        player = p2

    pos_row = int(input("Enter row (0-2): "))
    pos_col = int(input("Enter col (0-2): "))

    if tic_tack[pos_row][pos_col] != " ":
        print("Position already filled!")
        continue

    tic_tack[pos_row][pos_col] = player

    if check_win(player):
        print_board()
        print(f"{player} wins!")
        break

    player1_flag = not player1_flag
```

```
Bash

Player 1, choose X or O: X
[' ', ' ', ' ']
[' ', ' ', ' ']
[' ', ' ', ' ']

Player 1 Turn
Enter row (0-2): 0
Enter col (0-2): 0

Player 2 Turn
Enter row (0-2): 1
Enter col (0-2): 0

Player 1 Turn
Enter row (0-2): 0
Enter col (0-2): 1

Player 2 Turn
Enter row (0-2): 1
Enter col (0-2): 1

Player 1 Turn
Enter row (0-2): 0
Enter col (0-2): 2

['X', 'X', 'X']
['O', 'O', ' ']
[' ', ' ', ' ']

X wins!
```

14) Map Coloring:

```
graph = {
    'R1': ['R2','R3','R4'],
    'R2': ['R1','R3'],
    'R3': ['R1','R4','R5'],
    'R4': ['R1','R3','R5'],
    'R5': ['R3','R4']
}

colors = ['Red','Green','Blue']
assignment = {}

def solve(region_list):
    if not region_list:
        return True

    region = region_list[0]

    for color in colors:
        safe = True

        for neighbour in graph[region]:
            if neighbour in assignment and
assignment[neighbour] == color:
                safe = False
                break

        if safe:
```

```
                assignment[region] = color

                if solve(region_list[1:]):
                    return True

                del assignment[region]

        return False

regions = list(graph.keys())

if solve(regions):
    print("Coloring Solution: ")
    for r in assignment:
        print(r, "->", assignment[r])
else:
    print("No solution")
```

```
</> Bash

Coloring Solution:
R1 -> Red
R2 -> Green
R3 -> Blue
R4 -> Green
R5 -> Red
```

15) Wumpus :

```
world = []

# Create grid
for i in range(4):
    r = []
    for j in range(4):
        r.append(".")
    world.append(r)
```

```
# Static setup
world[2][0] = "P"
world[2][2] = "W"
world[3][1] = "G"
world[0][0] = "A"

def show():
    for row in world:
        print(row)
    print()

def is_safe(x, y):
    if world[x][y] == "P":
        print(f"⚠ ({x},{y}) has PIT!")
        return False
    if world[x][y] == "W":
        print(f"⚠ ({x},{y}) has WUMPUS!")
        return False
    return True

def move(ax, ay, nx, ny):
    print(f"\nTrying to move from ({ax},{ay}) →
({nx},{ny})")

    if is_safe(nx, ny):
        print("✅ Safe to move")

        world[ax][ay] = "."
```

```
            if world[nx][ny] == "G":
                world[nx][ny] = "AG"
            else:
                world[nx][ny] = "A"

            return nx, ny
        else:
            print("❌ Move blocked!")
            return ax, ay


print("Initial World:")
show()

# Step-by-step (YOUR path, but with checking)

ax, ay = 0, 0

# Step 1
ax, ay = move(ax, ay, 1, 0)
show()

# Step 2
ax, ay = move(ax, ay, 2, 0)    # blocked (Pit)
ax, ay = move(ax, ay, 1, 1)    # safe
show()
```

```python
# Step 3
ax, ay = move(ax, ay, 2, 1)
show()

# Step 4
ax, ay = move(ax, ay, 2, 2)    # blocked (Wumpus)
ax, ay = move(ax, ay, 3, 1)    # Gold
show()

print("   Gold Found!")
```

```
Bash

Initial World:
['A', '.', '.', '.']
['.', '.', '.', '.']
['P', '.', 'W', '.']
['.', 'G', '.', '.']

Trying to move from (0,0) → (1,0)
✅ Safe to move

Trying to move from (1,0) → (2,0)
⚠️ (2,0) has PIT!
❌ Move blocked!

Trying to move from (1,0) → (1,1)
✅ Safe to move

🎉 Gold Found!
```

16) Simple Facts

```
class KnowledgeBase:
    def __init__(self):
        self.facts = []
        self.rules = []

    # Add fact
    def add_fact(self, predicate, *args):
        self.facts.append((predicate, args))

    # Add rule
    def add_rule(self, head, body):
        self.rules.append((head, body))

    # Query
    def query(self, predicate, *args):
        results = []

        # Check facts
        for pred, fact_args in self.facts:
            if pred == predicate:
                subst = self.unify(args, fact_args)
                if subst is not None:
                    results.append(subst)

        # Check rules
        for head, body in self.rules:
            head_pred, *head_args = head
```

```
            if head_pred != predicate:
                continue

            subst = self.unify(args, head_args)
            if subst is not None:
                results += self.solve_body(body,
subst)

        return results

    # Solve rule body
    def solve_body(self, body, subst):
        if not body:
            return [subst]

        first, *rest = body
        pred, *args = first

        new_args = [subst.get(a, a) for a in args]

        results = []
        sub_results = self.query(pred, *new_args)

        for sub in sub_results:
            merged = subst.copy()
            merged.update(sub)
            results += self.solve_body(rest,
merged)
```

```
            return results

    # Unification
    def unify(self, query_args, fact_args):
        subst = {}
        for q, f in zip(query_args, fact_args):
            if q.isupper():  # variable
                subst[q] = f
            elif q != f:
                return None
        return subst


# Helper to parse input
def parse_input(text):
    parts = text.strip().split()
    return parts[0], parts[1:]


# ---------------- MAIN PROGRAM ----------------
kb = KnowledgeBase()

while True:
    print("\n--- MENU ---")
    print("1. Add Fact")
    print("2. Add Rule")
    print("3. Show Facts")
```

```
    print("4. Query")
    print("5. Exit")

    choice = input("Enter choice: ")

    # -------- ADD FACT --------
    if choice == "1":
        text = input("Enter fact (e.g., Parent John
Mary): ")
        pred, args = parse_input(text)
        kb.add_fact(pred, *args)
        print("Fact added!")

    # -------- ADD RULE --------
    elif choice == "2":
        print("Enter rule head (e.g., Grandparent X
Z):")
        head_text = input()
        head_pred, head_args =
parse_input(head_text)

        body = []
        n = int(input("Enter number of conditions:
"))
        for i in range(n):
            cond = input(f"Condition {i+1}: ")
            pred, args = parse_input(cond)
            body.append((pred, *args))
```

```
            kb.add_rule((head_pred, *head_args), body)
            print("Rule added!")

        # -------- SHOW FACTS --------
        elif choice == "3":
            print("\nFacts:")
            for fact in kb.facts:
                print(fact)

        # -------- QUERY --------
        elif choice == "4":
            text = input("Enter query (e.g., Parent
John X): ")
            pred, args = parse_input(text)
            result = kb.query(pred, *args)
            print("Result:", result)

        # -------- EXIT --------
        elif choice == "5":
            print("Exiting...")
            break

        else:
            print("Invalid choice!")
```

```
C:\Users\pavan\Documents\AI LAB>python simplefacts.py

--- MENU ---
1. Add Fact
2. Add Rule
3. Show Facts
4. Query
5. Exit
Enter choice: 1
Enter fact (e.g., Parent John Mary): PaidFees Sanjay
Fact added!

--- MENU ---
1. Add Fact
2. Add Rule
3. Show Facts
4. Query
5. Exit
Enter choice: 1
Enter fact (e.g., Parent John Mary): HasCertificate Sanjay
Fact added!

--- MENU ---
1. Add Fact
2. Add Rule
3. Show Facts
4. Query
5. Exit
Enter choice: 1
Enter fact (e.g., Parent John Mary): PaidFees Pavan
Fact added!

--- MENU ---
1. Add Fact
2. Add Rule
3. Show Facts
4. Query
5. Exit
Enter choice: 2
Enter rule head (e.g., Grandparent X Z):
Eligible X
Enter number of conditions: 2
Condition 1: PaidFees X
Condition 2: HasCertificate X
Rule added!
```

```
--- MENU ---
1. Add Fact
2. Add Rule
3. Show Facts
4. Query
5. Exit
Enter choice: 4
Enter query (e.g., Parent John X): Eligible X
Result: [{'X': 'Sanjay'}]
```

17) Monkey-Banana Problem:

```
def monkey_banana(monkey_pos, box_pos, banana_pos,
on_box, has_banana):

    print("Initial State:")
    print(monkey_pos, box_pos, on_box, has_banana)

    # Step 1: Move monkey to box
    if monkey_pos != box_pos:
        print("Monkey moves to box location")
        monkey_pos = box_pos

    # Step 2: Push box to banana
    if box_pos != banana_pos:
        print("Monkey pushes box to banana
location")
        box_pos = banana_pos
        monkey_pos = banana_pos

    # Step 3: Climb box
    if not on_box:
        print("Monkey climbs the box")
        on_box = True

    # Step 4: Grab banana
    if on_box and monkey_pos == banana_pos:
        print("Monkey grabs the banana")
        has_banana = True
```

```
    print("Final State:")
    print(monkey_pos, box_pos, on_box, has_banana)


# Example call
monkey_banana("A", "B", "C", False, False)
```

```
C:\Users\pavan\Documents\AI LAB>python monkeybanana.py
Initial State:
A B False False
Monkey moves to box location
Monkey pushes box to banana location
Monkey climbs the box
Monkey grabs the banana
Final State:
C C True True
```